Created by: Leonard Estrada
Date Created: April 28, 2026

Introduction

Purpose

This documentation defines the collaboration rules and application lifecycle management (ALM) standards for all Power Apps developers working across the HEMY environments.

Why These Rules Exist

Multiple developers work on Model-driven apps within the same shared environments. Without clear rules, the following problems will occur:

- **Silent overwrites** — The last developer to save a form wins. Previous changes are lost without warning.
- **Orphaned components** — Tables, columns, and flows created outside of solutions become unmanageable.
- **Broken deployments** — Unmanaged solutions imported into UAT or PROD create dependency issues that are difficult to reverse.
- **Untraceable changes** — Without naming conventions and ownership, no one knows who changed what or why.

Environment Overview

The HEMY platform uses four environments arranged in a promotion pipeline:

```
HEMY EXPLORATION ──► HEMY DEV ──► HEMY UAT ──► HEMY PROD
   (Prototype)      (Build)      (Test)      (Live)
```

- **HEMY EXPLORATION** — Sandbox for testing feasibility. Everything here is disposable.
- **HEMY DEV** — The primary development environment. All production-bound work starts here inside proper solutions.
- **HEMY UAT** — Receives managed solution imports from DEV. Used for testing only. No direct changes allowed.
- **HEMY PROD** — The live environment. Only managed solutions that have passed UAT testing are deployed here.

Scope

These rules apply to all developers building or modifying:

- Model-driven apps
- Tables, columns, and relationships
- Forms, views, and dashboards
- Business rules and classic workflows
- Cloud flows (Power Automate)
- Security roles
- Plugins and custom code
- Canvas apps embedded in Model-driven apps

Getting Started

This page walks new developers through the initial setup required before starting work in the HEMY environments.

Prerequisites

- Access to all four HEMY environments (EXPLORATION, DEV, UAT, PROD)
- Membership in the designated Teams channel for development coordination
- Access to the shared repository for solution backups (SharePoint or Azure DevOps)
- Access to the Form Lock Register

First-Time Setup in HEMY DEV

1. Verify the Publisher

Before creating any components, confirm that the **HEMY publisher** is available in HEMY DEV:

Setting	Value
Display Name	Hemy (HEMY)
Name	HEMY
Prefix	`hmy`

If the publisher does not exist, contact the team lead. **Do not create your own publisher** and never use the default publisher.

2. Create Your Solution

Create a new unmanaged solution for your feature or module:

- **Display Name:** `HEMY <Your Initials>` (e.g., `HEMY LE`)
- **Name:** `HEMY_<Initials>` (e.g., `HEMY_WellPlanning_JD`)
- **Publisher:** Select the HEMY publisher (`hemy`)
- **Version:** Start at `1.0.0.0`

3. Familiarize Yourself with Existing Components

Before creating new components:

1. Review the existing tables, forms, and flows in the environment.
2. Check other devlopers' solutions to understand what is already being worked on.
3. Ask in the team channel if you are unsure whether a component already exists.

4. Consult with NS|LE about the development and before doing anything.

Before Your First Day of Work

1. Read all pages in this documentation, especially:
    - Environment Guidelines
    - Form Development
    - Naming Conventions
2. Bookmark the Form Lock Register.
3. Join the designated Teams channel and introduce yourself.
4. Confirm your access to the shared solution backup repository.

Checklist

- [ ] I have access to all four HEMY environments
- [ ] I have joined the Teams development channel
- [ ] I have access to the Form Lock Register
- [ ] I have access to the shared solution backup repository
- [ ] I have verified the HEMY publisher exists in DEV
- [ ] I have created my personal unmanaged solution in DEV
- [ ] I have read all pages in this documentation

Environment Guidelines

Each environment has a specific purpose and set of rules. Developers must respect the boundaries of each environment.

HEMY EXPLORATION

Purpose: Proof of concept and feasibility testing.

Rules

- Developers may experiment freely without formal solution structure.
- Treat everything in this environment as **disposable**. Do not build production-quality logic here.
- **Document what worked** (screenshots, notes, configuration details) before anything is cleaned up or abandoned.
- Abandoned POCs must be cleaned up monthly to avoid clutter.
- Findings and validated concepts from EXPLORATION must be **rebuilt from scratch** in HEMY DEV — do not export unmanaged solutions from EXPLORATION into DEV.

What Is Allowed

- Creating tables, forms, and flows for testing purposes
- Rapid prototyping without naming conventions
- Testing third-party connectors and integrations

What Is NOT Allowed

- Treating EXPLORATION as the development environment
- Exporting solutions from EXPLORATION directly into UAT or PROD

HEMY DEV

Purpose: Primary development environment where validated concepts are properly built.

Rules

- All work must be done inside a **named unmanaged solution** (see Solution Strategy).
- Use the standard **publisher prefix** (`hemy_`). Never use the default `new_` prefix.
- Do not create tables, columns, or any components outside of a solution.
- Do not delete or rename another developer's components without their approval.
- Publish your customizations immediately after completing a unit of work. Unpublished changes are invisible to others and create confusion.
- Follow all Naming Conventions.

What Is Allowed

- Creating and modifying components inside your own unmanaged solution
- Adding shared components to your solution (after checking the Form Lock Register)
- Exporting managed solutions for deployment to UAT

What Is NOT Allowed

- Working in the Default Solution
- Creating components without adding them to a solution
- Directly modifying another developer's solution without coordination

---

HEMY UAT

Purpose: Testing environment. Receives managed solution imports from DEV.

Rules

- **No direct changes are allowed in UAT.** All changes arrive via managed solution import from DEV.
- Only testers and the **designated deployment lead** operate in this environment.
- Bugs found during UAT testing are fixed in **HEMY DEV**, re-exported, and re-imported to UAT.
- Never hotfix directly in UAT.

What Is Allowed

- Importing managed solutions from DEV
- Functional testing and user acceptance testing
- Logging bugs and feedback for developers to fix in DEV

What Is NOT Allowed

- Creating or modifying any components directly
- Importing unmanaged solutions
- Applying quick fixes or workarounds directly in UAT

HEMY PROD

Purpose: Live production environment.

Rules

- **Strictly managed solutions only.** No unmanaged customizations, ever.
- Deployments require **sign-off from UAT testing** before import.
- Only the **designated deployment lead** (or CI/CD pipeline) imports solutions.

- A **deployment log** must be maintained: who deployed what, when, and which solution version.

What Is Allowed

- Importing managed solutions that have passed UAT sign-off
- Monitoring and observing application behavior

What Is NOT Allowed

- Any direct customization or component creation
- Importing unmanaged solutions
- Skipping UAT and deploying directly from DEV
- Any developer deploying without deployment lead authorization

Solution Strategy

Solutions are the foundation of organized development in Power Apps. Every developer must follow these rules to avoid conflicts and ensure clean deployments.

Core Rule

Every developer must work inside their own unmanaged solution in HEMY DEV. Never customize components directly in the Default Solution.

Solution Structure

Per-Developer Solutions

Each developer creates their own unmanaged solution for the feature or module they are working on.

Naming format: `HEMY_<DevInitials>`

Examples:

- `HEMY_LE`

Master Release Solution

When features are ready for deployment, components are consolidated into a single **master unmanaged solution** before exporting. These Solutions are the Main solutions to be deployed using Pipelines

Naming format: `Name_of_Solution_v<Major>.<Minor>.<Patch>.<Build>`

Examples:

- `HEMY_P61 Budget Reporting_v1.0.0.1`
  - Each Deployment will automatically update the Build #
- `HEMY_P61 Budget Reporting_v1.2.0.0`
  - A new solution is recommended to be created when there is a major change.

Publisher

All developers must use the **same publisher** to ensure consistent prefixes across all components.

Setting	Value
Display Name	Hemy
Name	HEMY

Setting	Value
Prefix	`hmy`

> **Moving forward, the use of the default publisher** (`new_` prefix) is not allowed. Components created with the wrong prefix cannot be easily renamed and will cause long-term maintenance issues.

Before Starting Work

Before adding a component to your solution, check the following:

1. **Does the component already exist** in the environment? If so, another developer may own it.
2. **Is the component already in another developer's solution?** If yes, coordinate with that developer before adding it to yours.
3. **Is a form or shared component locked?** Check the Form Lock Register.

Solution Export Rules

- **Export as managed** when deploying to UAT and PROD.
- **Version your solutions** using semantic versioning (`Major.Minor.Patch.Build`). Never reuse a version number.
- Store exported solution `.zip` files in the **shared repository** (SharePoint or Azure DevOps) with the version number and export date.
- Include release notes describing what changed in each version.

No Duplicate Components Across Solutions

A component must exist in only one developer's solution at any given time. If a component already lives in another solution, it must not be added to yours.

Why This Matters

When the same component (e.g., Case Main Form) exists in multiple solutions, deploying those solutions creates unpredictable results. The last solution imported wins, silently overwriting changes from the other. This makes deployments unreliable and changes untraceable.

Rule

Scenario	Action
Component exists in Solution A	Do **not** add it to Solution B

Scenario	Action
You need to modify a component in another solution	Coordinate with the owner of that solution
A component must move between solutions	Remove it from the original solution first, then add it to the new one

Enforcement

The **Deployment Lead** validates every solution before export. If duplicate components are found across solutions, the deployment will be **rejected** until the duplication is resolved. See the [Deployment Lead]() page for the full validation checklist.

Example

> Developer A has `Case Main Form` in `HEMY_CaseManagement_DA`.
>
> Developer B needs to add fields to the same form. Developer B must **not** add `Case Main Form` to `HEMY_Escalation_DB`. Instead, Developer B coordinates with Developer A, or the form is transferred to Developer B's solution after being removed from Developer A's.

Component Ownership

- The developer who creates a component is its **owner** and is responsible for its maintenance.
- Do not modify, rename, or delete another developer's components without their explicit approval.
- If a component needs to be shared across multiple developer solutions, agree on a single owner who is responsible for changes to that component.

Form Development

Form conflicts are the **highest risk area** when multiple developers share an environment. Power Apps does not merge form changes — the last save wins and silently overwrites the other developer's work.

The Golden Rule

Never have two developers editing the same form at the same time.

Form Lock Register

Maintain a shared **Form Lock Register** (Excel sheet, Teams list, or Teams channel) where a developer "checks out" a form before editing and "checks in" when done.

Register Format

Form Name	Table	Locked By	Since	Expected Release	Purpose
Case Main Form	incident	JPT	2026-04-28	2026-04-28 EOD	Adding approval tab
Hard Asset Main Form	msdyn_customerasset	KMA	2026-04-28	2026-04-29	New required fields

Lock Rules

1. **Before editing any form**, check the register. If the form is locked by another developer, do not edit it.
2. **Add your lock entry** before you begin editing.
3. **Release your lock** as soon as you finish editing and publish. Do not hold locks overnight unless necessary.
4. At **end of day**, release any locks you still hold. Post a summary of changes in the team channel.
5. If a form has been locked for more than **2 business days**, the team lead will review and resolve.

If a Form Is Locked

When the form you need is locked by another developer:

- **Option 1:** Work on something else and wait for the lock to be released.
- **Option 2:** Coordinate with the lock holder to batch your changes together.
- **Option 3:** Create a **secondary form** for prototyping your layout and field placement. Merge your sections into the main form once the lock is released.

Form Design Best Practices

Split Large Forms

Where possible, break a large form into **multiple forms** so that different developers can own different forms on the same table. For example:

- `Case - Main Form` (owned by Developer A)
- `Case - Operations Form` (owned by Developer B)
- `Case - Quick Create` (owned by Developer C)

Use Tabs for Separation

If a single form must contain many sections, organize by **tabs**. Assign tab ownership so developers know which tabs they are responsible for. Communicate this ownership in the team channel or register.

Publish Immediately

After completing your form edits, **publish customizations immediately**. Unpublished form changes are invisible to other developers and will cause confusion when they open the form editor.

What NOT to Do

- Do not edit a form that is locked by another developer.
- Do not hold form locks for more than 2 business days without team lead approval.
- Do not leave customizations unpublished at end of day.
- Do not delete tabs or sections created by another developer without their approval.
- Do not rearrange fields on a tab owned by another developer.

Naming Conventions

Consistent naming makes components traceable, reduces confusion, and ensures smooth deployments. All developers must follow these conventions in **HEMY DEV**.

Summary Table

Asset	Convention	Example
Solution (per-developer)	`HEMY_<Initials>`	`HEMY_LE`
Solution (release)	`HEMY_SolutionName_v<Version>`	`HEMY_P29 Ordering Process_v1.2.0.0`
Publisher prefix	`hmy_`	`hmy_`
Table	`hmy_<entityname>`	`hmy_new_table`
Column	`hmy_<columnname>`	`hemy_new_column`
Form	`<Table Display Name> - <Purpose>`	`Case - Main Form`
View	`<Table Display Name> - <Description>`	`Case - Active Cases by`
Business Rule	`<Initials> - <Description>`	`LE - Set Purchase Order Status to Paid`
Classic Workflow	`HEMY - <Module> - <Action>`	`HEMY - Cases - Fetch Owner Business from Customer`
Cloud Flow	`HEMY - <Module> - <Action>`	`HEMY - Purchase Orders - Send Email on Approval`
Web Resource	`hmy_/scripts/<name>.js`	`hmy_/scripts/hardasset_form.js`

Rules

1. **Always use the `hmy_` publisher prefix.** Never use the default `new_` prefix.
2. **Table and column names** use lowercase with no spaces. Use descriptive names that reflect the business purpose.
3. **Display names** use proper casing and spaces for readability.
4. **Business rules and workflows** must include the developer's initials so changes can be traced back to the responsible developer.
5. **Cloud flows** must start with `HEMY` followed by the module name for easy filtering and identification.

6. **Never rename** an existing component's schema name after creation — this breaks dependencies. If a name is wrong, coordinate with the team before taking action.
7. **Avoid abbreviations** in display names unless they are universally understood within the team (e.g., `UAT`, `API`).

Deployment Lead

The Deployment Lead is the gatekeeper for all solution deployments across HEMY environments. No solution moves to UAT or PROD without the Deployment Lead's validation and approval.

Responsibilities

1. **Validate solutions** before export from HEMY DEV.
2. **Import managed solutions** to HEMY UAT and HEMY PROD.
3. **Reject deployments** that fail validation checks.
4. **Maintain the deployment log** for all imports.
5. **Coordinate rollbacks** when a deployment causes issues.

Pre-Deployment Validation Checklist

Before exporting any solution from HEMY DEV, the Deployment Lead must complete the following checks:

1. No Duplicate Components Across Solutions

This is the most critical check. Each component must exist in exactly one solution. If the same object appears in multiple solutions, the deployment is rejected.

How to check:

1. Open each developer's solution in HEMY DEV.
2. Compare the component lists across all active solutions.
3. Flag any component that appears in more than one solution.

Common duplicates to watch for:

Component Type	Example
Forms	Case Main Form in both Solution A and Solution B
Views	Active Cases view in both Solution A and Solution C
Business Rules	Same rule added to two solutions
Cloud Flows	Same flow referenced in multiple solutions
Tables	Same table added to multiple solutions (check if only needed components are included)

> **Note:** A table being present in multiple solutions is acceptable only if each solution adds **different components** (e.g., Solution A adds Column X, Solution B adds Column Y). What is not acceptable is the **same component** (form, view, column) existing in both.

Resolution steps when duplicates are found:

1. Identify which developer owns the component.
2. Remove the component from all other solutions.
3. If both developers need to modify it, one developer takes ownership and the other coordinates changes through them.

For components already in multiple solutions

1. We don't need to delete them unless they are causing issues in HEMY PROD
2. Usually, Forms in multiple solution and have multiple Managed Layers in PRODUCTION causes Issues like misaligned fields or duplicated lookups. These are the ones we will fix.

2. Solution Checker Results

1. Run **Solution Checker** on the solution in HEMY DEV.
2. All **critical** and **high-severity** issues must be resolved before export.
3. Medium and low issues should be reviewed and documented if not resolved.

3. Version Number

1. Confirm the solution version has been incremented.
2. Confirm the version number has never been used before.
3. Version must follow the format: `Major.Minor.Patch.Build`.

4. Publisher Verification

1. Confirm the solution uses the **HEMY publisher** (`hmy` prefix).
2. Reject any solution using the default publisher (`new_` prefix).

5. Component Completeness

1. Verify all required components are included in the solution (no missing dependencies).
2. Check that no unnecessary components have been accidentally added.
3. Confirm no components from the Default Solution have been pulled in.

6. Release Notes

1. Confirm the developer has provided release notes describing what changed.
2. Release notes must list all added, modified, and removed components.

Validation Outcome

Result	Action
All checks pass	Proceed with managed export and import to UAT
Duplicate components found	**Reject.** Developer must remove duplicates before re-submitting.
Solution Checker critical issues	**Reject.** Developer must resolve issues.
Missing version increment	**Reject.** Developer must update version number.
Wrong publisher	**Reject.** Developer must recreate components with correct publisher.
Missing release notes	**Hold.** Developer must provide notes before deployment proceeds.

Deployment Execution

HEMY UAT

1. In HEMY DEV, open the solution to be deployed and navigate on Pipelines
2. In the Pipeline dropdown, select **DEV-to-UAT**
3. Click on the `Deploy Here` button
4. The Deploying Solution window will appear on the right side, you may select Now or Later for the Deployment Schedule. Then click Next
5. This will automatically validate the solution for missing dependencies and errors.
6. If the solution contains Connection References, Enable the **Re-establish connection to activate you solution** toggle, this will let you select which connection reference in UAT you will use for your flows.
   - For Dataverse connections you may use **Microsoft Dataverse - Shared**
   - For other connections are to be determined.
7. Write in the Deployment Notes.

HEMY PROD

1. Same process as in **HEMY UAT** but change step 2 to **DEV-to-PROD**

Deployment Schedule

To avoid conflicts and rushed validations:

- **UAT deployments:** Scheduled times only (e.g., Tuesday and Thursday mornings). No ad-hoc imports.
- **PROD deployments:** Only after UAT sign-off. Requires at minimum 1 business day notice.

- **Emergency hotfixes:** Must still go through DEV → UAT → PROD, but validation can be expedited with team lead approval.

What the Deployment Lead Must NOT Do

- Deploy without completing the validation checklist.
- Allow unmanaged solutions into UAT or PROD.
- Skip the duplicate component check.
- Deploy to PROD without written UAT sign-off.
- Make direct customizations in UAT or PROD to "fix" a deployment issue.

Deployment Flow

This page defines how solutions move from HEMY DEV through UAT and into PROD.

Pipeline Overview

```
HEMY DEV (unmanaged solutions per developer)
    │
    ├── Merge components into master unmanaged solution
    │
    ├── Export as MANAGED solution (.zip)
    │
    ├── Store .zip in shared repository with version number
    │
    ▼
HEMY UAT (import managed solution)
    │
    ├── Functional testing and user acceptance testing
    │
    ├── Bugs? → Fix in DEV → Re-export → Re-import to UAT
    │
    ├── Testing sign-off
    │
    ▼
HEMY PROD (import same managed solution)
    │
    └── Deployment log updated
```

Step-by-Step Process

1. Prepare in HEMY DEV

1. Verify all components are inside your unmanaged solution.
2. Test your changes in DEV to confirm basic functionality.

2. Preparing Master Solution in HEMY DEV

Developers

1. Inform the Deployment Lead about the developments done
2. List all the components to be added in the Master Solution.
3. Provide the link of all the component

Deployment Lead

1. Review the development and suggest improvements if necessary.
2. **Add to Solution** all the components provided by the Developers to the Master Solution.
3. Follow the Deployment Execution from Deployment Lead.

3. UAT Testing

1. Testers execute test cases against the new functionality.
2. Bugs and issues are logged and sent back to developers.
3. Developers fix bugs in **HEMY DEV**, increment the version, and repeat steps 2-3.
4. When all tests pass, the test lead provides **written sign-off**.

5. Deploy to HEMY PROD

1. Only after UAT sign-off is received.
2. Only the **designated deployment lead** deploys to PROD.
3. Follow the Deployment Execution from Deployment Lead.

Deployment Log

Maintain a deployment log (SharePoint list or Excel file) with the following fields:

Field	Description
Date	Deployment date and time
Solution Name	Full solution name with version
Deployed By	Name of the deployment lead
Environment	Target environment (UAT or PROD)
Sign-off By	Name of the person who authorized the deployment
Release Notes	Summary of changes or link to detailed notes
Status	Success / Failed / Rolled Back

Rollback

If a deployment to PROD causes issues:

1. **Do not attempt to fix directly in PROD.**
2. Import the **previous managed solution version** to roll back.
3. Investigate and fix the issue in DEV.
4. Follow the full deployment pipeline again (DEV → UAT → PROD).

What NOT to Do

- Do not import unmanaged solutions into UAT or PROD.
- Do not skip UAT and deploy directly from DEV to PROD.
- Do not reuse version numbers.
- Do not deploy without written UAT sign-off.
- Do not re-export from DEV after UAT sign-off — use the same `.zip` that was tested.

Daily Collaboration Protocol

These are the day-to-day practices every developer must follow to avoid conflicts in the shared environments.

Start of Day

1. **Check the Form Lock Register** for any active locks on forms you plan to work on.
2. **Read the team channel** for announcements, ongoing deployments, or warnings from other developers.
3. **Plan your work** and announce what components you intend to modify today.

Before Editing a Shared Component

1. **Announce in the team channel** which component you are about to edit (form, view, business rule, flow, etc.).
2. **Wait for acknowledgment** if another developer recently worked on the same component.
3. **Lock the form** in the Form Lock Register if you are editing a form (see Form Development).
4. **Check the component** in the form editor or solution to confirm no unpublished changes from another developer are pending.

During Development

1. Work inside **your own unmanaged solution** at all times.
2. Save and publish frequently — small incremental publishes are safer than large batches.
3. If you discover a conflict or unexpected change to a component, **stop and notify the team** before proceeding.
4. Follow all Naming Conventions.

After Completing a Unit of Work

1. **Publish your customizations immediately.** Unpublished changes are invisible to other developers.
2. **Release any form locks** you no longer need.
3. **Test your changes** in DEV to confirm basic functionality.
4. Notify the team channel if your changes affect shared components.

End of Day

1. **Publish all pending customizations.** Do not leave unpublished changes overnight.

2. **Release all form locks** you hold. If you must hold a lock overnight, notify the team with an explanation.

3. **Post a brief summary** of what you changed today in the team channel. Example:

> "Updated Case Main Form — added Approval tab with 3 new fields. Updated business rule for auto-status. Lock released."

Communication Expectations

- Use the designated **Teams channel** for all development announcements.
- **Respond promptly** when another developer asks about a component you own or have locked.
- If you will be absent, **release all locks** and notify the team of any in-progress work that others may need to continue.

Weekly Sync

Hold a brief weekly sync meeting (15-30 minutes) to:

- Review active development work and upcoming changes
- Identify potential conflicts before they happen
- Plan deployments to UAT
- Clean up abandoned solutions or components in DEV and EXPLORATION

Conflict Resolution

Even with good practices, conflicts will happen. This page defines how to identify, resolve, and prevent recurring conflicts.

Accidental Overwrites

If two developers accidentally modify the same component (form, view, business rule, etc.):

1. The **second developer to publish** must verify that the first developer's changes are still intact.
2. If changes were overwritten, restore from the **solution zip backup** — this is why versioned exports are critical.
3. The two developers must coordinate to merge both sets of changes manually.
4. Update the Form Lock Register or team channel to prevent the same conflict from happening again.

Disputed Ownership

If two developers disagree about who should own or modify a component:

1. Check the Naming Conventions — the developer whose initials are in the component name is the designated owner.
2. If ownership is unclear, the **team lead** decides.
3. Never resolve a dispute by overwriting the other developer's work.

Broken Deployments

If a managed solution import to UAT or PROD fails:

1. **Do not attempt to fix the issue in UAT or PROD.**
2. Review the import error log to identify the cause.
3. Common causes:
    - Missing dependency (a component references something that does not exist in the target environment)
    - Version conflict (an older version imported over a newer one)
    - Schema mismatch (a column type was changed in DEV but the target still has the old type)
4. Fix the issue in **HEMY DEV**, increment the solution version, and re-export.
5. Re-import the corrected solution to UAT for testing.

Unmanaged Components in UAT or PROD

If unmanaged customizations are discovered in UAT or PROD:

1. **Do not delete them immediately** — they may be supporting active functionality.
2. Identify who created them and why.

3. Recreate the component properly in HEMY DEV inside a solution.
4. Deploy the managed version to UAT/PROD, which will overlay the unmanaged component.
5. Document the incident and reinforce the rule: no direct customizations in UAT or PROD.

Escalation Path

Situation	First Action	Escalate To
Form overwrite	Restore from backup, coordinate with other developer	Team Lead
Ownership dispute	Check naming conventions	Team Lead
Failed deployment	Review error log, fix in DEV	Deployment Lead
Unmanaged component in UAT/PROD	Identify owner, recreate in DEV	Team Lead
Repeated rule violations	Peer discussion	Team Lead / Management

Prevention

The best conflict resolution is prevention:

- **Use the Form Lock Register** consistently.
- **Announce your work** in the team channel before starting.
- **Publish frequently** so others can see your changes.
- **Export solution backups** regularly so overwrites can be reversed.
- **Attend the weekly sync** to surface potential conflicts early.